

Guide Complet — Projet Solaire : Architecture PL/pgSQL-Centric (PostgreSQL + Python)

Objectif final : Convertir le système existant (SQL Server + pyodbc + architecture en couches Python "Repository/Service/Fenêtre") en une architecture où **toute la logique métier vit dans PostgreSQL (fonctions et procédures PL/pgSQL)**, et où **Python ne fait plus que deux choses : appeler ces fonctions, et afficher le résultat (Tkinter)**.

C'est l'inverse du pattern MVC/Repository classique : au lieu de "Python qui pense, PostgreSQL qui stocke", on passe à "PostgreSQL qui pense ET qui stocke, Python qui demande et qui montre".

Table des matières

1. Comprendre le système — fonctionnement détaillé
 2. Pourquoi changer d'architecture (MVC → PL/pgSQL-centric)
 3. Tutoriel PostgreSQL / PL/pgSQL (rappel + approfondissement)
 4. Tutoriel Python nécessaire (psycopg2 + Tkinter, version "client mince")
 5. **Tutoriel : faire dialoguer Python et PL/pgSQL proprement**
 6. Migration étape par étape vers l'architecture PL/pgSQL-centric
 7. Structure finale du projet
 8. Comment modifier et étendre le code
 9. Checklist de migration
-

1. Comprendre le système — fonctionnement détaillé

1.1 C'est quoi ce projet ?

C'est un outil de dimensionnement de système solaire pour une maison à Madagascar. L'utilisateur entre ses appareils électriques (réfrigérateur, TV, lampes...) avec leurs horaires d'utilisation et leur puissance. Le système calcule ensuite :

- Quelle taille de panneau solaire acheter
- Quelle capacité de batterie prévoir
- Quel convertisseur (onduleur) choisir
- Combien d'énergie le panneau produit vs consomme
- Le gain financier par rapport au réseau électrique (Jirama)

1.2 Les tranches horaires

La journée est divisée en 3 tranches :

Tranche	Heures	Signification
Matin	06h00 → 17h00	Ensoleillement plein, panneau produit à 100%

Tranche	Heures	Signification
Fin-Après-midi	17h00 → 19h00	Soleil déclinant, panneau produit à 50% du pic
Soir	19h00 → 06h00	Nuit, panneau ne produit rien, batterie prend le relais

Ces tranches sont stockées dans la base de données (table `tranche_horaire`) et sont modifiables.

1.3 Les heures de pointe

La table `heure_pointe` définit des plages horaires où le prix du kWh est plus cher (majoration en %). Par exemple :

- Jour ouvrable 12h-14h : +35% sur le prix de base
- Jour ouvrable 17h-19h : +35% sur le prix de base
- Weekend 12h-14h : +40% sur le prix de base

Cela permet de calculer le gain financier réel par rapport au réseau Jirama.

1.4 La Configuration (rendement)

La table `Configuration` stocke le rendement (en %) de chaque source d'énergie par tranche horaire :

- Panneau solaire le matin → 40% (ex: un panneau de 1000W donne 400W nets)
- Panneau solaire l'après-midi → 50% (du rendement matin, donc $50\% \times 40\% = 20\%$ nets)
- Batterie la nuit → 50% (pertes lors de la charge/décharge)

Ces pourcentages sont configurables par l'utilisateur.

1.5 Le flux de saisie (Session)

Chaque utilisation crée une session :

1. L'utilisateur clique "Nouvelle Session" → une entrée `session_input` est créée en base
2. Il saisit ses appareils un par un (nom, heure de début, heure de fin, puissance en Watts)
3. Il clique "Valider en base" → les appareils sont enregistrés dans la table `Input`
4. La fenêtre de résultats s'ouvre automatiquement avec les calculs

1.6 Les calculs métier

Ce sont **exactement les mêmes calculs** que dans le projet d'origine — la seule chose qui change, c'est **où** ils sont exécutés (PL/pgSQL au lieu de Python). Liste des calculs à porter :

`get_pic_puissance_matin_aprem()`

Parcourt toutes les minutes de 6h à 19h, calcule la puissance instantanée (somme des appareils allumés à chaque minute), retourne le maximum (pic). C'est la puissance que le panneau doit pouvoir fournir en permanence.

`suggerer_panneau_solaire()`

Pour chaque minute de 6h à 19h :

- Calcule la puissance demandée par les appareils
- Divise par le rendement (40% le matin, 20% l'après-midi)
- Retourne le maximum → c'est la puissance brute du panneau à acheter

Exemple : si à 10h les appareils consomment 800W et le rendement est 40%, il faut un panneau de $800/0.40 = 2000W$.

getBatteriePratique()

- Calcule l'énergie totale (Wh) consommée par tous les appareils du soir (19h → 6h)
- Applique le pourcentage de perte de la batterie (ex: +50% pour les pertes)
- C'est la capacité de batterie à acheter

Exemple : 500Wh consommés la nuit + 50% de pertes = batterie de 750Wh.

getConvertisseure()

Pic de puissance sur 24h × 2. Le convertisseur (onduleur) doit pouvoir absorber les pics, avec un facteur de sécurité ×2.

getReste()

Énergie produite par le panneau moins énergie consommée (pendant les heures d'ensoleillement). Si positif : excédent (peut être revendu ou stocké). Si négatif : le panneau est sous-dimensionné.

get_cout_ouvrable() / get_cout_weekend()

Calcule combien coûterait (en Ariary) l'énergie non produite par le solaire sur le réseau Jirama, en tenant compte des heures de pointe. Représente le gain journalier du système solaire.

1.7 Les tables de la base de données

```

session_input ← Une session = une saisie d'appareils
    ↓ (1 session → N appareils)
Input          ← Chaque appareil (nom, heure de début/fin, puissance)

tranche_horaire ← Définit Matin / Fin-Après-midi / Soir
    ↓ (utilisé par)
Configuration  ← Rendement (%) par tranche + par source d'énergie

energie        ← Types d'énergie (Panneau solaire, Batterie, etc.)

jour           ← Types de jours (Ouvrable, Weekend)
    ↓ (1 jour → N prix, N heures de pointe)
prix_jour      ← Prix du kWh par type de jour
heure_pointe   ← Plages horaires majorées + pourcentage de majoration

```

PROF

2. Pourquoi changer d'architecture (MVC → PL/pgSQL-centric)

2.1 L'ancienne architecture (rappel)

Interface (fenetre/) ↔ Service (service/) ↔ Repository (repository/)
↔ Base de données

- **Repository** : ne fait que du SQL (INSERT, SELECT, UPDATE, DELETE). Aucune logique.
- **Service** : logique métier en Python. Utilise les repositories.
- **Fenêtre** : affichage. Utilise les services.

C'est le pattern Repository / couches, très utile quand on veut pouvoir **changer de base de données** facilement (on ne touche que les repositories).

2.2 La nouvelle architecture (PL/pgSQL-centric)

Interface (fenetre/) —appelle—> Fonctions/Procédures PL/pgSQL (dans PostgreSQL)
(Tkinter, affichage pur) (TOUTE la logique métier + les données)

Il n'y a plus de couche Repository ni de couche Service en Python. Python devient un **client mince (thin client)** :

1. Python ouvre une connexion psycopg2
2. Python appelle une fonction/procédure PostgreSQL avec des paramètres
3. PostgreSQL fait **tout le calcul** (boucles sur les minutes, agrégations, jointures, règles de gestion) et renvoie un résultat déjà prêt
4. Python **affiche** ce résultat dans Tkinter, point final

2.3 Avantages et inconvénients (à connaître pour la soutenance)

Avantages :

- Toute la logique est centralisée à un seul endroit (la base), donc cohérente quel que soit le client (Tkinter aujourd'hui, mais demain une appli web ou mobile pourraient réutiliser exactement les mêmes fonctions sans dupliquer la logique).
- Les calculs s'exécutent **là où sont les données** : pas besoin de rapatrier des milliers de lignes en Python pour les agréger, le moteur SQL le fait plus efficacement.
- Moins de code Python à maintenir, moins de couches, code plus court.
- Les règles de gestion sont protégées : même un script SQL externe mal écrit ne peut pas contourner les règles si elles sont dans des fonctions/triggers.

Inconvénients (à mentionner pour montrer l'esprit critique) :

- Dépendance forte à PostgreSQL : si on change de SGBD, il faut réécrire toute la logique (pas seulement les requêtes).
- PL/pgSQL est moins pratique que Python pour du code complexe (pas de vraies structures de données riches, débogage plus difficile).
- Tests unitaires plus difficiles à écrire qu'en Python.

- Toute l'équipe doit savoir lire/écrire du PL/pgSQL, pas seulement du Python.

C'est un compromis : on choisit la centralisation et la performance contre la portabilité et la facilité de test. C'est un choix d'architecture défendable, pas "la seule bonne solution".

3. Tutoriel PostgreSQL / PL/pgSQL

3.1 Différences SQL Server → PostgreSQL

SQL Server	PostgreSQL
IDENTITY(1,1)	SERIAL ou GENERATED ALWAYS AS IDENTITY
OUTPUT INSERTED.id	RETURNING id
NVARCHAR(255)	VARCHAR(255)
FLOAT	FLOAT8 ou DOUBLE PRECISION
DATETIME	TIMESTAMP
GETDATE()	NOW()
DECLARE @var INT; SET @var = ...	var INT := ...; (dans un bloc DO)
SCOPE_IDENTITY()	RETURNING id dans l'INSERT
? (paramètre pyodbc)	%s (paramètre psycopg2)
LOWER(col) = ?	LOWER(col) = %s

3.2 Créer les tables en PostgreSQL

```
-- Supprimer les tables si elles existent (ordre important à cause des
FK)
DROP TABLE IF EXISTS input CASCADE;
DROP TABLE IF EXISTS configuration CASCADE;
DROP TABLE IF EXISTS prix_jour CASCADE;
DROP TABLE IF EXISTS heure_pointe CASCADE;
DROP TABLE IF EXISTS session_input CASCADE;
DROP TABLE IF EXISTS tranche_horaire CASCADE;
DROP TABLE IF EXISTS energie CASCADE;
DROP TABLE IF EXISTS jour CASCADE;

-- Tables de référence
CREATE TABLE tranche_horaire (
    id SERIAL PRIMARY KEY,
    heure_debut TIME NOT NULL,
    heure_fin TIME NOT NULL,
    libelle VARCHAR(255) NOT NULL
);
```

```

CREATE TABLE energie (
    id SERIAL PRIMARY KEY,
    nom VARCHAR(255) NOT NULL
);

CREATE TABLE configuration (
    id SERIAL PRIMARY KEY,
    id_tranche_horaire INT NOT NULL REFERENCES tranche_horaire(id),
    id_energie INT NOT NULL REFERENCES energie(id),
    pourcentage FLOAT8 NOT NULL
);

CREATE TABLE jour (
    id SERIAL PRIMARY KEY,
    type_jour VARCHAR(255) NOT NULL
);

CREATE TABLE prix_jour (
    id SERIAL PRIMARY KEY,
    id_jour INT NOT NULL REFERENCES jour(id),
    prix FLOAT8 NOT NULL
);

CREATE TABLE heure_pointe (
    id SERIAL PRIMARY KEY,
    id_jour INT NOT NULL REFERENCES jour(id),
    heure_debut TIME NOT NULL,
    heure_fin TIME NOT NULL,
    pourcentage FLOAT8 NOT NULL
);

CREATE TABLE session_input (
    id SERIAL PRIMARY KEY,
    date_creation TIMESTAMP NOT NULL
);

-- IMPORTANT : on renomme "Input" en "input" (minuscule) pour éviter
-- tout problème de casse en PostgreSQL (voir section 8.8)
CREATE TABLE input (
    id SERIAL PRIMARY KEY,
    nom_appareil VARCHAR(255) NOT NULL,
    heure_debut TIMESTAMP NOT NULL,
    heure_fin TIMESTAMP NOT NULL,
    puissance FLOAT8 NOT NULL,
    id_session_input INT NOT NULL REFERENCES session_input(id)
);

```

PROF

3.3 PL/pgSQL — Blocs et fonctions (rappel des bases)

PL/pgSQL est le langage procédural de PostgreSQL. On peut écrire des fonctions, des procédures, et des blocs anonymes.

Bloc anonyme (s'exécute une fois, pas réutilisable) :

```
DO $$
DECLARE
    v_session_id INT;
BEGIN
    INSERT INTO session_input (date_creation) VALUES (NOW())
    RETURNING id INTO v_session_id;

    RAISE NOTICE 'Session créée avec ID : %', v_session_id;

    INSERT INTO input (nom_appareil, heure_debut, heure_fin, puissance,
id_session_input)
    VALUES ('TV', '1900-01-01 08:00:00', '1900-01-01 12:00:00', 55.0,
v_session_id);
END;
$$;
```

Fonction qui retourne un scalaire :

```
CREATE OR REPLACE FUNCTION get_prix_jour(p_type_jour VARCHAR)
RETURNS FLOAT8 AS $$
DECLARE
    v_prix FLOAT8;
BEGIN
    SELECT p.prix INTO v_prix
    FROM prix_jour p
    JOIN jour j ON p.id_jour = j.id
    WHERE LOWER(j.type_jour) = LOWER(p_type_jour);

    RETURN COALESCE(v_prix, 0); -- COALESCE = valeur par défaut si NULL
END;
$$ LANGUAGE plpgsql;

-- Appel :
SELECT get_prix_jour('Ouvrable');
```

Fonction qui retourne une table :

```
CREATE OR REPLACE FUNCTION get_appareils_par_session(p_session_id INT)
RETURNS TABLE(
    nom_appareil VARCHAR,
    heure_debut TEXT,
    heure_fin TEXT,
    puissance FLOAT8
) AS $$
BEGIN
```

```

RETURN QUERY
SELECT
    i.nom_appareil,
    TO_CHAR(i.heure_debut, 'HH24:MI:SS'),
    TO_CHAR(i.heure_fin, 'HH24:MI:SS'),
    i.puissance
FROM input i
WHERE i.id_session_input = p_session_id;
END;
$$ LANGUAGE plpgsql;

-- Appel :
SELECT * FROM get_appareils_par_session(1);

```

Procédure stockée (sans retour) :

```

CREATE OR REPLACE PROCEDURE supprimer_session(p_id INT)
LANGUAGE plpgsql AS $$
BEGIN
    DELETE FROM input WHERE id_session_input = p_id;
    DELETE FROM session_input WHERE id = p_id;
END;
$$;

-- Appel :
CALL supprimer_session(3);

```

3.4 Données initiales PostgreSQL

```

INSERT INTO tranche_horaire (heure_debut, heure_fin, libelle) VALUES
('06:00:00', '17:00:00', 'Matin'),
('17:00:00', '19:00:00', 'Fin-Après-midi'),
('19:00:00', '06:00:00', 'Soir');

INSERT INTO energie (nom) VALUES
('Panneau solaire'),
('Batterie');

INSERT INTO configuration (id_tranche_horaire, id_energie, pourcentage)
VALUES
(1, 1, 40.0),
(2, 1, 50.0),
(3, 2, 50.0);

INSERT INTO jour (type_jour) VALUES ('Ouvrable'), ('Weekend');

INSERT INTO prix_jour (id_jour, prix) VALUES
(1, 190.0),

```



```
(2, 210.0);

INSERT INTO heure_pointe (id_jour, heure_debut, heure_fin, pourcentage)
VALUES
(1, '12:00:00', '14:00:00', 35.0),
(1, '17:00:00', '19:00:00', 35.0),
(2, '12:00:00', '14:00:00', 40.0),
(2, '17:00:00', '19:00:00', 40.0);
```

3.5 Structures avancées indispensables pour porter TOUTE la logique en PL/pgSQL

Pour migrer `logicService.py` en PL/pgSQL, il faut connaître quelques outils en plus des fonctions simples :

Boucles (FOR, LOOP) — utile pour "parcourir chaque minute de 6h à 19h" comme dans `get_pic_puissance_matin_aprem()` :

```
DO $$
DECLARE
    v_minute TIME := '06:00:00';
    v_pic FLOAT8 := 0;
    v_puissance_instant FLOAT8;
BEGIN
    WHILE v_minute < '19:00:00' LOOP
        -- calculer la puissance instantanée à v_minute (exemple
simplifié)
        v_puissance_instant := 0; -- ... somme des appareils allumés ici
        IF v_puissance_instant > v_pic THEN
            v_pic := v_puissance_instant;
        END IF;
        v_minute := v_minute + INTERVAL '1 minute';
    END LOOP;
    RAISE NOTICE 'Pic : %', v_pic;
END;
$$;
```

PROF

generate_series — l'astuce PL/pgSQL pour remplacer la boucle "minute par minute"

efficacement. Plutôt que de boucler ligne par ligne en PL/pgSQL (lent), on génère la série de minutes directement en SQL :

```
SELECT m AS minute
FROM generate_series(
    '1900-01-01 06:00:00'::timestamp,
    '1900-01-01 18:59:00'::timestamp,
    INTERVAL '1 minute'
) AS m;
```

Combiné à une jointure avec **input**, on calcule la puissance instantanée par minute en une seule requête (voir section 6.5 pour l'exemple complet de **suggerer_panneau_solaire**).

Types composites / RECORD — pour retourner plusieurs valeurs à la fois (ex: panneau + batterie + convertisseur + reste, en un seul appel) :

```
CREATE TYPE resultat_dimensionnement AS (  
    puissance_panneau FLOAT8,  
    capacite_batterie FLOAT8,  
    puissance_convertisseur FLOAT8,  
    reste FLOAT8,  
    cout_ouvrable FLOAT8,  
    cout_weekend FLOAT8  
);
```

EXCEPTION — gestion des erreurs en PL/pgSQL, équivalent du **try/except** Python :

```
CREATE OR REPLACE FUNCTION diviser_securise(p_a FLOAT8, p_b FLOAT8)  
RETURNS FLOAT8 AS $$  
BEGIN  
    RETURN p_a / p_b;  
EXCEPTION  
    WHEN division_by_zero THEN  
        RETURN 0;  
END;  
$$ LANGUAGE plpgsql;
```

3.6 Conseils PostgreSQL essentiels

- **Sensibilité à la casse** : PostgreSQL met les noms de tables/colonnes en minuscule par défaut. Pour éviter les problèmes, utilise tout en minuscule (d'où le renommage **Input** → **input**).
- **SERIAL vs IDENTITY** : **SERIAL** est plus ancien mais plus courant. Les deux fonctionnent.
- **Transactions** : tout est transactionnel.

```
BEGIN;  
INSERT INTO session_input (date_creation) VALUES (NOW());  
INSERT INTO input (...) VALUES (...);  
COMMIT; -- ou ROLLBACK; en cas d'erreur
```

Vérifier ce qui existe :

```
\dt          -- Lister les tables  
\d input     -- Décrire une table  
\df          -- Lister les fonctions
```

4. Tutoriel Python nécessaire (client mince)

4.1 Connexion avec psycopg2

```
pip install psycopg2-binary
```

```
import psycopg2

conn = psycopg2.connect(
    host="localhost",
    database="solaire",
    user="postgres",
    password="ton_mot_de_passe",
    port=5432
)

conn.close()
```

4.2 Appeler une fonction PL/pgSQL depuis Python

C'est la différence majeure avec l'ancien projet : Python n'écrit (presque) plus de **SELECT ... FROM ... WHERE ... JOIN**, il appelle des fonctions toutes faites.

Fonction scalaire :

```
cursor = conn.cursor()
cursor.execute("SELECT get_prix_jour(%s)", ("Ouvrable",))
prix = cursor.fetchone()[0]
print(prix)
```

Fonction qui retourne une table :

```
cursor.execute("SELECT * FROM get_appareils_par_session(%s)", (1,))
rows = cursor.fetchall()
for nom, debut, fin, puissance in rows:
    print(nom, debut, fin, puissance)
```

Procédure (sans retour, via **CALL**) :

```
cursor.execute("CALL supprimer_session(%s)", (3,))
conn.commit()
```

Fonction qui retourne un type composite (plusieurs valeurs en un appel) :

```
cursor.execute("SELECT * FROM calculer_dimensionnement(%s)",
(id_session,))
row = cursor.fetchone()
puissance_panneau, capacite_batterie, puissance_convertisseur, reste,
cout_ouvrable, cout_weekend = row
```

4.3 Accéder aux colonnes par nom

```
import psycopg2.extras

cursor = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
cursor.execute("SELECT * FROM get_appareils_par_session(%s)", (1,))
rows = cursor.fetchall()
for row in rows:
    print(row['nom_appareil'])
```

4.4 Tkinter — rappel des bases

```
import tkinter as tk
from tkinter import ttk, messagebox

root = tk.Tk()
root.title("Mon titre")
root.geometry("800x600")

label = tk.Label(root, text="Bonjour")
label.pack()

entry = tk.Entry(root)
entry.pack()

def cliquer():
    valeur = entry.get()
    messagebox.showinfo("Résultat", f"Tu as tapé : {valeur}")

btn = tk.Button(root, text="Valider", command=cliquer)
btn.pack()

root.mainloop()
```

Mise en page avec grid :

```
tk.Label(root, text="Nom appareil:").grid(row=0, column=0, padx=5,
pady=5)
entry_nom = tk.Entry(root)
entry_nom.grid(row=0, column=1, padx=5, pady=5)
```

Tableau (Treeview) :

```
colonnes = ("id", "nom", "puissance")
table = ttk.Treeview(root, columns=colonnes, show="headings")
table.heading("id", text="ID")
table.heading("nom", text="Appareil")
table.heading("puissance", text="Puissance (W)")
table.column("id", width=50)
table.column("nom", width=200)
table.column("puissance", width=100)
table.pack(fill=tk.BOTH, expand=True)

table.insert("", "end", values=(1, "Réfrigérateur", 120))

for row in table.get_children():
    table.delete(row)
```

Combobox :

```
combo = ttk.Combobox(root, state="readonly")
combo['values'] = ["Option 1", "Option 2", "Option 3"]
combo.pack()
valeur = combo.get()
combo.current(0)
```

PROF

Fenêtre secondaire :

```
def ouvrir_fenetre_secondaire():
    fenetre = tk.Toplevel(root)
    fenetre.title("Résultats")
    tk.Label(fenetre, text="Voici les résultats").pack()
```

4.5 Gestion des erreurs

```
try:
    cursor.execute("CALL ajouter_appareil(%s, %s, %s, %s, %s)",
```

```

        (id_session, "Climatiseur", "22:00:00", "06:00:00",
1500))
    conn.commit()
except psycopg2.Error as e:
    conn.rollback() # IMPORTANT : annuler en cas d'erreur
    messagebox.showerror("Erreur", str(e))

```

Important : avec psycopg2, si une requête échoue, la connexion est dans un état d'erreur jusqu'à ce qu'on fasse `conn.rollback()`. Sans rollback, toutes les requêtes suivantes échoueront aussi.

5. Tutoriel : faire dialoguer Python et PL/pgSQL proprement

C'est la partie centrale de ce guide. Voici les patterns à connaître pour bâtir une application où **Python n'est qu'un client** de la logique PL/pgSQL.

5.1 Le principe général

Côté PostgreSQL (le "cerveau")	Côté Python (les "mains et les yeux")
Stocke les données	N'a aucune variable métier persistante
Contient toutes les règles de calcul (PL/pgSQL)	Ne fait aucun calcul métier
Expose des fonctions/procédures comme une API	Appelle ces fonctions comme on appellerait une API REST
Valide les données (contraintes, triggers)	Affiche le résultat, gère les widgets, capture les saisies

Règle d'or : si tu te poses la question "est-ce que je mets ce `if`/cette boucle/ce calcul en Python ou en SQL ?", la réponse dans ce projet est **toujours en PL/pgSQL**. Python ne doit contenir aucune règle de gestion (pas de seuils, pas de pourcentages, pas de formules).

PROF

5.2 Les 4 types d'appels Python → PostgreSQL

1. Fonction scalaire (un seul résultat) → utiliser `SELECT fonction(...)`

```

cursor.execute("SELECT get_pic_puissance(%s)", (id_session,))
pic = cursor.fetchone()[0]

```

2. Fonction tabulaire (plusieurs lignes) → utiliser `SELECT * FROM fonction(...)`

```

cursor.execute("SELECT * FROM get_historique_sessions()")
sessions = cursor.fetchall()

```

3. Fonction qui retourne un type composite (plusieurs valeurs en une fois, comme un "objet") → idéal pour remplacer un appel qui en Python aurait nécessité 6 appels séparés (`getPanneau()`, `getBatterie()`, etc.)

```
cursor.execute("SELECT * FROM calculer_dimensionnement(%s)",
(id_session,))
resultat = cursor.fetchone()
# resultat est un tuple/dict avec toutes les valeurs calculées par
Postgres
```

4. Procédure (action, pas de valeur de retour, ex: insérer/supprimer) → `CALL`

```
cursor.execute("CALL creer_session_avec_appareils(%s, %s)",
(date_creation, json_appareils))
conn.commit()
```

5.3 Passer une liste d'appareils en un seul appel (au lieu de N appels)

Problème classique : l'utilisateur saisit 5 appareils dans Tkinter, il ne faut pas faire 5 allers-retours réseau. Solution : sérialiser en JSON côté Python, et laisser PostgreSQL désérialiser et boucler en PL/pgSQL.

Côté PL/pgSQL :

```
CREATE OR REPLACE FUNCTION creer_session_avec_appareils(p_appareils
JSON)
RETURNS INT AS $$
DECLARE
    v_session_id INT;
    v_appareil JSON;
BEGIN
    INSERT INTO session_input (date_creation) VALUES (NOW())
    RETURNING id INTO v_session_id;

    FOR v_appareil IN SELECT * FROM json_array_elements(p_appareils)
    LOOP
        INSERT INTO input (nom_appareil, heure_debut, heure_fin,
puissance, id_session_input)
        VALUES (
            v_appareil->>'nom',
            (v_appareil->>'heure_debut')::timestamp,
            (v_appareil->>'heure_fin')::timestamp,
            (v_appareil->>'puissance')::float8,
            v_session_id
        );
    END LOOP;

    RETURN v_session_id;
```

```
END;  
$$ LANGUAGE plpgsql;
```

Côté Python :

```
import json  
  
appareils = [  
    {"nom": "Réfrigérateur", "heure_debut": "1900-01-01 00:00:00",  
     "heure_fin": "1900-01-01 23:59:00", "puissance": 120},  
    {"nom": "TV", "heure_debut": "1900-01-01 08:00:00", "heure_fin":  
     "1900-01-01 12:00:00", "puissance": 55},  
]  
  
cursor.execute("SELECT creer_session_avec_appareils(%s)",  
               (json.dumps(appareils),))  
id_session = cursor.fetchone()[0]  
conn.commit()
```

C'est le pattern à privilégier dès qu'il y a "plusieurs lignes à insérer/traiter d'un coup" : un seul appel réseau, toute la boucle se fait côté serveur.

5.4 Récupérer un résultat complexe (objet "résultat de dimensionnement") en un seul appel

C'est l'équivalent PL/pgSQL de "construire un objet Python `RésultatDimensionnement` avec tous les champs calculés".

```
DROP TYPE IF EXISTS resultat_dimensionnement CASCADE;  
CREATE TYPE resultat_dimensionnement AS (  
    puissance_panneau FLOAT8,  
    capacite_batterie FLOAT8,  
    puissance_convertisseur FLOAT8,  
    reste FLOAT8,  
    cout_ouvrable FLOAT8,  
    cout_weekend FLOAT8  
);  
  
CREATE OR REPLACE FUNCTION calculer_dimensionnement(p_session_id INT)  
RETURNS resultat_dimensionnement AS $$  
DECLARE  
    v_resultat resultat_dimensionnement;  
BEGIN  
    v_resultat.puissance_panneau :=  
suggerer_panneau_solaire(p_session_id);  
    v_resultat.capacite_batterie := get_batterie_pratique(p_session_id);  
    v_resultat.puissance_convertisseur :=  
get_convertisseur(p_session_id);
```



```

v_resultat.reste := get_reste(p_session_id);
v_resultat.cout_ouvrable := get_cout_ouvrable(p_session_id);
v_resultat.cout_weekend := get_cout_weekend(p_session_id);

RETURN v_resultat;
END;
$$ LANGUAGE plpgsql;

```

```

cursor.execute("SELECT * FROM calculer_dimensionnement(%s)",
(id_session,))
row = cursor.fetchone()
panneau, batterie, convertisseur, reste, cout_ouvrable, cout_weekend =
row

# Python ne fait QUE de l'affichage à partir d'ici
tk.Label(frame, text=f"Panneau : {panneau:.0f} W").pack()
tk.Label(frame, text=f"Batterie : {batterie:.0f} Wh").pack()

```

5.5 Validation des données : où la mettre ?

Deux niveaux possibles, à utiliser ensemble (défense en profondeur) :

1. **Contraintes SQL** (niveau base) : **CHECK**, **NOT NULL**, clés étrangères → empêchent les données invalides d'exister, quel que soit le client.

```

ALTER TABLE input ADD CONSTRAINT puissance_positive CHECK (puissance >
0);

```

2. **Validation légère côté Python** (niveau formulaire) : juste pour donner un message d'erreur immédiat et convivial avant même d'envoyer la requête (ex: champ vide). Ce n'est **pas** de la logique métier, c'est de l'ergonomie — donc ça reste acceptable en Python.

```

if not entry_nom.get().strip():
    messagebox.showwarning("Erreur", "Le nom de l'appareil est
obligatoire")
    return

```

5.6 Gérer les erreurs métier renvoyées par PostgreSQL

PL/pgSQL peut lever des erreurs explicites avec **RAISE EXCEPTION**, que Python intercepte :

```

CREATE OR REPLACE FUNCTION ajouter_appareil(p_session_id INT, p_nom
VARCHAR, p_puissance FLOAT8)

```

```

RETURNS INT AS $$
DECLARE
    v_id INT;
BEGIN
    IF p_puissance <= 0 THEN
        RAISE EXCEPTION 'La puissance doit être positive (reçu: %)',
p_puissance;
    END IF;

    INSERT INTO input (nom_appareil, puissance, id_session_input,
heure_debut, heure_fin)
    VALUES (p_nom, p_puissance, p_session_id, NOW(), NOW())
    RETURNING id INTO v_id;

    RETURN v_id;
END;
$$ LANGUAGE plpgsql;

```

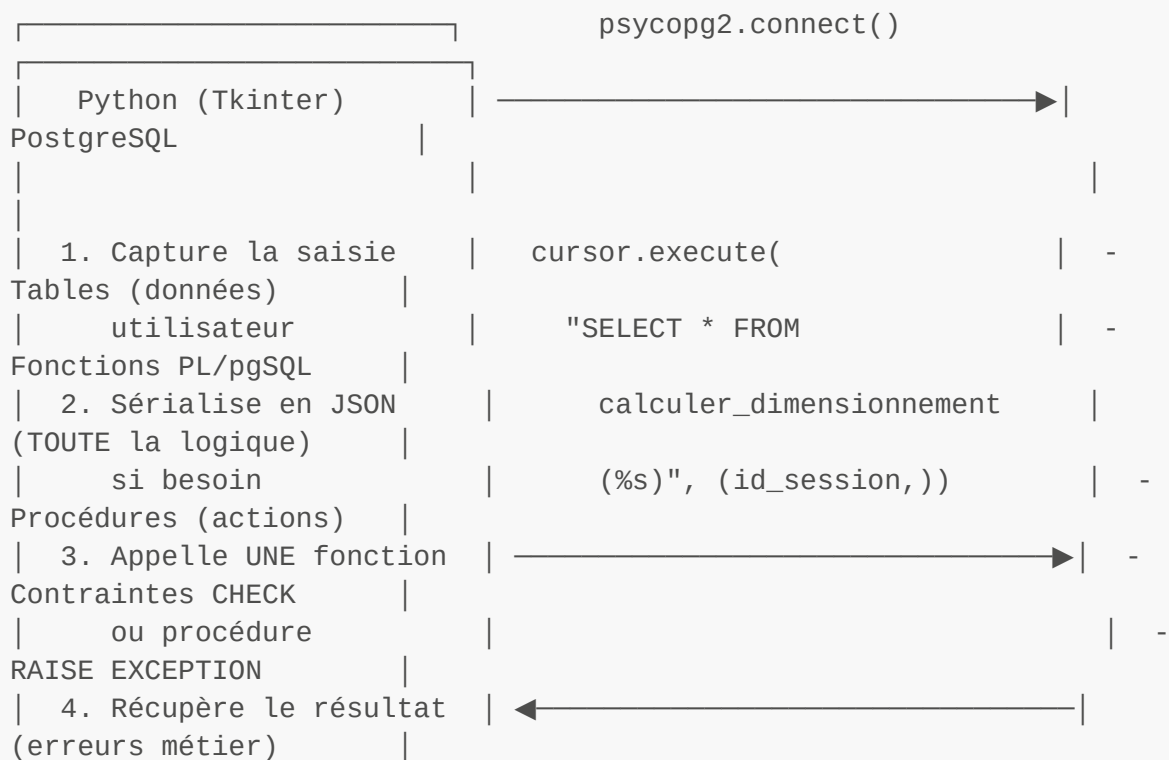
```

try:
    cursor.execute("SELECT ajouter_appareil(%s, %s, %s)", (id_session,
"Lampe", -10))
    conn.commit()
except psycopg2.errors.RaiseException as e:
    conn.rollback()
    messagebox.showerror("Erreur métier", str(e))

```

5.7 Schéma récapitulatif du dialogue Python ↔ PL/pgSQL

PROF



déjà calculé	row = cursor.fetchone()	
5. Affiche dans Tkinter		

6. Migration étape par étape vers l'architecture PL/pgSQL-centric

Étape 0 — Installer PostgreSQL

1. Télécharger PostgreSQL sur <https://www.postgresql.org/download/>
2. Installer avec pgAdmin (interface graphique incluse)
3. Retenir le mot de passe du compte **postgres** lors de l'installation
4. Créer une base de données nommée **solaire** dans pgAdmin

Étape 1 — Installer psycopg2

```
pip install psycopg2-binary
```

Étape 2 — Créer le schéma

Exécuter le SQL de la section 3.2 (tables) puis les données initiales de la section 3.4, dans un fichier **sql/01_schema.sql** et **sql/02_data.sql**.

Étape 3 — Écrire connect/connect.py (client mince)

```
import psycopg2

DB_CONFIG = {
    "host": "localhost",
    "database": "solaire",
    "user": "postgres",
    "password": "VOTRE_MOT_DE_PASSE",
    "port": 5432
}

def get_connection():
    return psycopg2.connect(**DB_CONFIG)

conn = get_connection()
```

Étape 4 — Écrire toutes les fonctions PL/pgSQL métier

Dans un fichier `sql/03_fonctions_metier.sql`, on porte chaque calcul de `logicService.py` en PL/pgSQL. Exemple complet pour `suggerer_panneau_solaire` en utilisant `generate_series` (voir 3.5) :

```
CREATE OR REPLACE FUNCTION suggerer_panneau_solaire(p_session_id INT)
RETURNS FLOAT8 AS $$
DECLARE
    v_rendement_matin FLOAT8;
    v_rendement_aprem FLOAT8;
    v_max FLOAT8 := 0;
BEGIN
    SELECT pourcentage / 100.0 INTO v_rendement_matin
    FROM configuration
    WHERE id_tranche_horaire = 1 AND id_energie = 1;

    SELECT pourcentage / 100.0 INTO v_rendement_aprem
    FROM configuration
    WHERE id_tranche_horaire = 2 AND id_energie = 1;

    SELECT MAX(puissance_minute) INTO v_max
    FROM (
        SELECT
            CASE
                WHEN m::time < '17:00:00' THEN
                    COALESCE(SUM(i.puissance), 0) /
NULLIF(v_rendement_matin, 0)
                ELSE
                    COALESCE(SUM(i.puissance), 0) /
NULLIF(v_rendement_aprem, 0)
            END AS puissance_minute
        FROM generate_series(
            '1900-01-01 06:00:00'::timestamp,
            '1900-01-01 18:59:00'::timestamp,
            INTERVAL '1 minute'
        ) AS m
        LEFT JOIN input i
            ON i.id_session_input = p_session_id
            AND m >= i.heure_debut AND m < i.heure_fin
        GROUP BY m
    ) sous_requete;

    RETURN COALESCE(v_max, 0);
END;
$$ LANGUAGE plpgsql;
```

Les autres fonctions (`get_pic_puissance_matin_aprem`, `get_batterie_pratique`, `get_convertisseur`, `get_reste`, `get_cout_ouvrable`, `get_cout_weekend`) se construisent sur le même principe : `generate_series` + jointure avec `input` + agrégation, puis application des règles de

`configuration/heure_pointe/prix_jour`. Toutes ces fonctions sont ensuite assemblées dans `calculer_dimensionnement()` (voir section 5.4).

Étape 5 — Écrire les procédures CRUD (remplacent les repositories)

```
CREATE OR REPLACE PROCEDURE modifier_prix_jour(p_id_jour INT,
p_nouveau_prix FLOAT8)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE prix_jour SET prix = p_nouveau_prix WHERE id_jour =
p_id_jour;
END;
$$;

CREATE OR REPLACE PROCEDURE ajouter_tranche_horaire(p_debut TIME, p_fin
TIME, p_libelle VARCHAR)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO tranche_horaire (heure_debut, heure_fin, libelle)
VALUES (p_debut, p_fin, p_libelle);
END;
$$;
```

Étape 6 — Réécrire les fichiers `fenetre/*.py` en client mince

Plus de `service/` ni de `repository/`. Chaque fichier `fenetre/*.py` importe `connect.py` et appelle directement les fonctions SQL.

```
# fenetre/outputFenetre.py
import tkinter as tk
from connect.connect import conn

def afficher_resultats(id_session):
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM calculer_dimensionnement(%s)",
(id_session,))
    panneau, batterie, convertisseur, reste, cout_ouvrable, cout_weekend
= cursor.fetchone()

    fenetre = tk.Toplevel()
    fenetre.title("Résultats")

    tk.Label(fenetre, text=f"Panneau solaire : {panneau:.0f} W").pack()
    tk.Label(fenetre, text=f"Batterie : {batterie:.0f} Wh").pack()
    tk.Label(fenetre, text=f"Convertisseur : {convertisseur:.0f}
W").pack()
    tk.Label(fenetre, text=f"Reste (excédent/déficit) : {reste:.0f}
Wh").pack()
    tk.Label(fenetre, text=f"Gain jour ouvrable : {cout_ouvrable:.0f}
```

```
Ar").pack()
    tk.Label(fenetre, text=f"Gain weekend : {cout_weekend:.0f}")
Ar").pack()
```

Étape 7 — Tester

```
# test_connexion.py
from connect.connect import conn

cursor = conn.cursor()
cursor.execute("SELECT COUNT(*) FROM tranche_horaire")
print("Tranches:", cursor.fetchone()[0])

cursor.execute("SELECT * FROM calculer_dimensionnement(1)")
print("Dimensionnement session 1:", cursor.fetchone())

print("Connexion OK!")
```

7. Structure finale du projet (PL/pgSQL-centric)

```
solaire/
├── connect/
│   └── connect.py           ← Seule couche Python qui parle à la
base (psycopg2)
├── fenetre/                ← Interface utilisateur (Tkinter) –
affichage UNIQUEMENT
│   ├── Main.py             ← Menu principal
│   ├── inputFenetre.py     ← Saisie d'appareils (appelle
creer_session_avec_appareils)
│   ├── outputFenetre.py    ← Affiche le résultat de
calculer_dimensionnement()
│   ├── historique_session.py ← Affiche le résultat de
get_historique_sessions()
│   ├── CrudTrancheHoraire.py ← CRUD via procédures SQL
│   ├── CrudConfigurationFentre.py
│   ├── CrudHeurePointe.py
│   └── CrudPrixJour.py
├── sql/                    ← TOUTE la logique métier vit ici
│   ├── 01_schema.sql       ← Création des tables
│   ├── 02_data.sql         ← Données initiales
│   ├── 03_fonctions_metier.sql ← Tous les calculs (panneau, batterie,
coûts...)
│   ├── 04_procedures_crud.sql ← CRUD (créer session, ajouter appareil,
modifier prix...)
│   └── 05_types.sql        ← Types composites
```

```
(resultat_dimensionnement, etc.)
|
├─ test_connexion.py      ← Script de test rapide
└─ README.md             ← Comment installer / lancer le projet
```

Comparaison directe avec l'ancienne structure

Ancienne structure (MVC / Repository)	Nouvelle structure (PL/pgSQL-centric)
<code>modele/</code> (classes Python)	Plus nécessaire (les "objets" sont des types composites SQL ou de simples tuples)
<code>repository/*.py</code> (SQL en Python)	<code>sql/04_procedures_crud.sql</code> (le SQL vit dans la base)
<code>service/*.py</code> (logique métier Python)	<code>sql/03_fonctions_metier.sql</code> (la logique vit dans la base)
<code>fenetre/*.py</code> (affichage + appels services)	<code>fenetre/*.py</code> (affichage + appels SQL directs) — inchangé dans son rôle, juste branché différemment

Règle de placement d'un nouveau fichier : si ce que tu écris contient un calcul, une règle, un `if` métier, ou une boucle sur des données → ça va dans `sql/`. Si ce que tu écris affiche un widget, capture une saisie, ou appelle une fonction SQL → ça va dans `fenetre/`.

8. Comment modifier et étendre le code

8.1 Ajouter un nouvel appareil dans une session existante

```
from connect.connect import conn

cursor = conn.cursor()
cursor.execute(
    "SELECT ajouter_appareil(%s, %s, %s)",
    (2, "Climatiseur", 1500)
)
id_insere = cursor.fetchone()[0]
conn.commit()
print(f"Appareil ajouté avec l'ID {id_insere}")
```

8.2 Modifier le prix d'un jour

```
cursor.execute("CALL modifier_prix_jour(%s, %s)", (1, 200.0))
conn.commit()
```

8.3 Ajouter une nouvelle tranche horaire

```
cursor.execute("CALL ajouter_tranche_horaire(%s, %s, %s)", ("12:00:00",  
"14:00:00", "Midi"))  
conn.commit()
```

8.4 Ajouter un nouveau calcul métier

Tout se fait côté PL/pgSQL. Exemple : ajouter un calcul de consommation totale sur 24h.

```
CREATE OR REPLACE FUNCTION get_consommation_totale_24h(p_session_id INT)  
RETURNS FLOAT8 AS $$  
DECLARE  
    v_total FLOAT8;  
BEGIN  
    SELECT COALESCE(SUM(  
        puissance * EXTRACT(EPOCH FROM (heure_fin - heure_debut)) /  
3600.0  
    ), 0) INTO v_total  
    FROM input  
    WHERE id_session_input = p_session_id;  
  
    RETURN v_total;  
END;  
$$ LANGUAGE plpgsql;
```

Puis, côté Python, on l'ajoute simplement à l'affichage (aucune logique, juste un appel + un label) :

```
# Dans afficher_resultats(), après les calculs existants :  
cursor.execute("SELECT get_consommation_totale_24h(%s)", (id_session,))  
conso_totale = cursor.fetchone()[0]  
  
tk.Label(fenetre, text=f"Consommation totale 24h : {conso_totale:.2f}  
Wh",  
        font=("Arial", 11, "italic"), fg="red").pack(pady=5)
```

Remarque : on pourrait aussi l'intégrer directement dans le type `resultat_dimensionnement` et la fonction `calculer_dimensionnement()` pour ne faire qu'un seul appel réseau (recommandé si le calcul est toujours utilisé en même temps que les autres).

8.5 Ajouter un nouveau bouton dans le menu principal

```
# fenetre/Main.py  
def ouvrir_mon_ecran():
```



```
chemin = os.path.join(os.path.dirname(__file__),
"MonNouvelEcran.py")
subprocess.Popen([sys.executable, chemin])

btn_mon_ecran = tk.Button(root, text="Mon Nouvel Écran",
command=ouvrir_mon_ecran, width=30, pady=10)
btn_mon_ecran.pack(pady=10)
```

8.6 Créer une nouvelle fonctionnalité complète (ex: gestion de `type_panneau`)

Avec cette architecture, la structure est beaucoup plus courte qu'avant :

1. Créer la table SQL `type_panneau` dans `sql/01_schema.sql`
2. Créer les procédures CRUD (`creer_type_panneau`, `modifier_type_panneau`, `supprimer_type_panneau`) et une fonction `lister_type_panneau()` dans `sql/04_procedures_crud.sql`
3. Créer la fenêtre `fenetre/CrudTypePanneau.py` qui appelle directement ces 4 fonctions/procédures
4. Ajouter le bouton dans `fenetre/Main.py`

(Il n'y a plus d'étape "créer le modèle Python" ni "créer le repository" ni "créer le service" : c'est tout l'intérêt de cette architecture.)

8.7 Lancer l'application

```
# Depuis le dossier solaire/
python fenetre/Main.py
```

8.8 Problèmes courants et solutions

Problème	Cause	Solution
<code>psycopg2.OperationalError: could not connect</code>	PostgreSQL pas lancé ou mauvais mot de passe	Vérifier que PostgreSQL est démarré, vérifier les credentials dans <code>connect.py</code>
<code>InFailedSqlTransaction</code>	Requête précédente a échoué sans rollback	Ajouter <code>conn.rollback()</code> dans le bloc <code>except</code>
<code>column "x" does not exist</code>	Casse différente	Utiliser des noms en minuscule sans guillemets
<code>UndefinedTable: relation "Input" does not exist</code>	PostgreSQL a mis le nom en minuscule	Renommer la table <code>input</code> dans le SQL (recommandé : tout en minuscule)

Problème	Cause	Solution
Tkinter fenêtre ne s'affiche pas	Lancé depuis un environnement sans affichage	Lancer directement depuis un terminal, pas depuis SSH
<code>function calculer_dimensionnement(integer) does not exist</code>	Le type composite ou la fonction n'a pas été créé dans le bon ordre	Exécuter les scripts SQL dans l'ordre : <code>05_types.sql</code> avant <code>03_fonctions_metier.sql</code>
Le calcul met du temps pour de grosses sessions	Boucle PL/pgSQL ligne par ligne au lieu de <code>generate_series</code> + agrégation SQL	Réécrire la fonction en utilisant <code>generate_series</code> + <code>GROUP BY</code> (voir section 3.5 et 6)

Astuce table `input` : PostgreSQL convertit `Input` en `input` (minuscule). Si la table est créée avec `CREATE TABLE Input (...)`, il faudra écrire `"Input"` (avec guillemets) dans chaque requête. La solution la plus simple est de tout nommer en minuscule dès la création (`input`, comme fait dans ce guide).

9. Résumé rapide — Checklist de migration

- ☐ Installer PostgreSQL et créer la base `solaire`
- ☐ Installer `psycpg2-binary` via pip
- ☐ Créer `sql/01_schema.sql` (tables, tout en minuscule) et l'exécuter
- ☐ Créer `sql/02_data.sql` (données initiales) et l'exécuter
- ☐ Créer `sql/05_types.sql` (types composites comme `resultat_dimensionnement`)
- ☐ Créer `sql/03_fonctions_metier.sql` : porter chaque calcul de l'ancien `logicService.py` en fonction PL/pgSQL (utiliser `generate_series` pour les boucles minute par minute)
- ☐ Créer `sql/04_procedures_crud.sql` : porter chaque opération des anciens `repository/*.py` en procédure/fonction PL/pgSQL
- ☐ Écrire `connect/connect.py` (le seul point de contact Python ↔ PostgreSQL)
- ☐ Supprimer (ou ne pas créer) les dossiers `modele/`, `repository/`, `service/`
- ☐ Réécrire chaque fichier `fenetre/*.py` pour qu'il n'appelle QUE des fonctions/procédures SQL et ne fasse QUE de l'affichage
- ☐ Tester avec `python test_connexion.py`
- ☐ Lancer `python fenetre/Main.py` et vérifier chaque fonctionnalité
- ☐ Vérifier qu'aucun fichier Python ne contient de `if`/boucle/formule métier (seule la base PostgreSQL doit en contenir)

Guide généré à partir de l'analyse complète du projet solaire (ETU003922), réorienté vers une architecture où PostgreSQL/PL-pgSQL porte toute la logique métier et Python se limite à l'appel de fonctions et à l'affichage Tkinter.

